



Process Synchronization(2)

Dept. of Computer Science
Hanyang University





Classical Problems of Synchronization

- Bounded-Buffer Problem
- Readers and Writers Problem: **scheduling**
- Dining-Philosophers Problem: **deadlock**

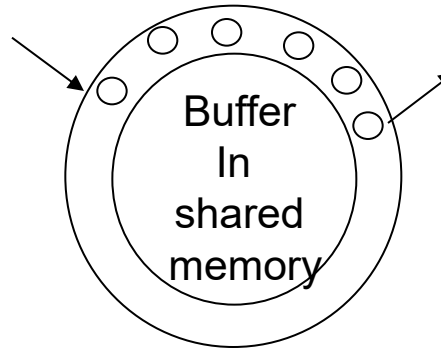
Bounded-Buffer: Shared-Memory Solution

Any empty buf?
Produce full buf

If yes, but ...

Can I access
shared data now?

Producer
in



Consumer
out

Any full buf?
Produce empty buf

If yes, but ...

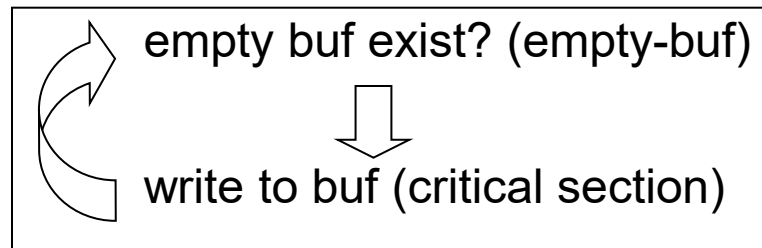
Can I access
shared data now?

Shared variable:	buf	==> Need binary semaphore
Resource count:	# of full buf # of empty buf	==> Need integer semaphore

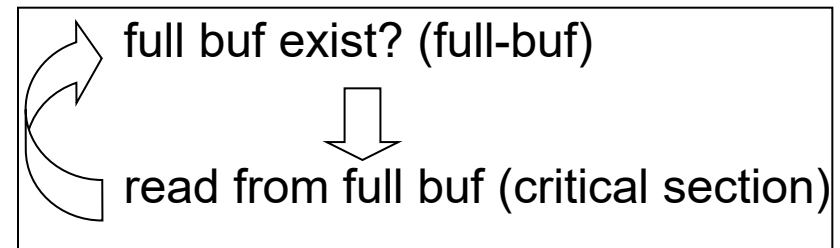
Bounded-Buffer: Shared-Memory Solution

Here N is integer semaphore - used for resource count

Producer



Consumer



integer semaphore

$N_{\text{empty-buf}}$

$N_{\text{full-buf}}$

binary semaphore

S_{mutex}

$P(N_{\text{empty-buf}})$

$P(S_{\text{mutex}})$

write to buf

$V(S_{\text{mutex}})$

$V(N_{\text{full-buf}})$

$P(N_{\text{full-buf}})$

$P(S_{\text{mutex}})$

read from buf

$V(S_{\text{mutex}})$

$V(N_{\text{empty-buf}})$

Bounded-Buffer Problem

Shared data

semaphore full = 0, empty = n, mutex = 1;

Producer

```
do {
    ...
    produce an item in nextp
    ...
    P(empty);
    P(mutex);
    ...
    add nextp to buffer
    ...
    V(mutex);
    V(full);
} while (1);
```

Consumer

```
do {
    ...
    P(full)
    P(mutex);
    ...
    remove an item from buffer to nextc
    ...
    V(mutex);
    V(empty);
    ...
    consume the item in nextc
    ...
} while (1);
```



Readers-Writers Problem

DB is accessed by many users

When one writes to DB - exclusive access (semaphore db)

Readers can access DB, concurrently !

Priority?

Waiting: N readers (int readcount, semaphore mutex), 1 writer

- OR
- A. Let all readers finish (concurrently), then writer start
 - B. Let writer finish (urgent), then readers start

Try A. solution first:

If one reader starts reading, let all other readers start read too.

If last reader finishes reading, let the writer start.

```
semaphore      mutex = 1,   db = 1; /* mutex is used for readcount */
int            readcount = 0
```

Reader: first reader -> try P(db), last reader -> do V(db)
 [readcount access] is critical section, surround it by P() V()

Readers-Writers Problem

Shared data

semaphore mutex = 1, db = 1;

int readcount = 0

Writer

P(db);

writing is performed

V(db);

Reader

P(mutex);

readcount++;

if (readcount == 1) **P(db);** /* first reader?*/

/* check for the writer */

V(mutex);

reading is performed

P(mutex);

readcount--;

if (readcount == 0) **V(db);** /* last reader?*/

/*enable writer*/

V(mutex);

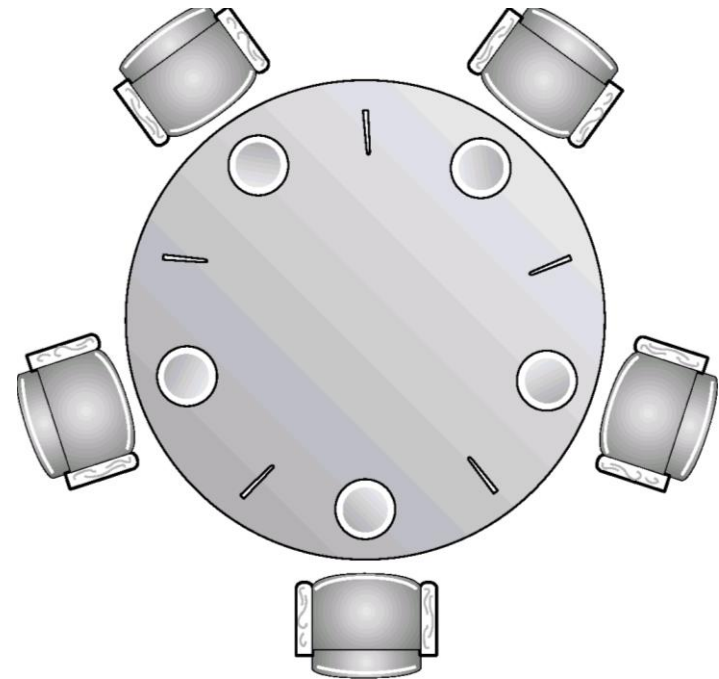
Dining-Philosophers Problem: Think & Eat Repeatedly

Shared data

```
semaphore chopstick[5];  
// Initially all values are 1
```

Philosopher i

```
do {  
    P(chopstick[i])  
    P(chopstick[(i+1) % 5])  
    ...  
    eat  
    ...  
    V(chopstick[i]);  
    V(chopstick[(i+1) % 5]);  
    ...  
    think  
    ...  
} while (1);    ==> deadlock may occur!
```



Deadlock and Starvation

- **Deadlock** – two or more processes are *waiting indefinitely* for an event that can be caused by only one of the waiting processes

- Let S and Q be two semaphores initialized to 1

P_0	P_1	
P(S);	P(Q);	hold one
P(Q);	P(S);	wait for another one
⋮	⋮	
V(S);	V(Q);	release here
V(Q)	V(S);	

- **Starvation** – *indefinite blocking*. A process may never be removed from the semaphore queue in which it is suspended.
 - Semaphore with LIFO queue



Deadlock - Remedies?

- Pick up only both : should be performed in the CS
- Asymmetric coding
 - Odd philosopher - try Left first
 - Even - try Right first
- Allow at most 4 to sit together

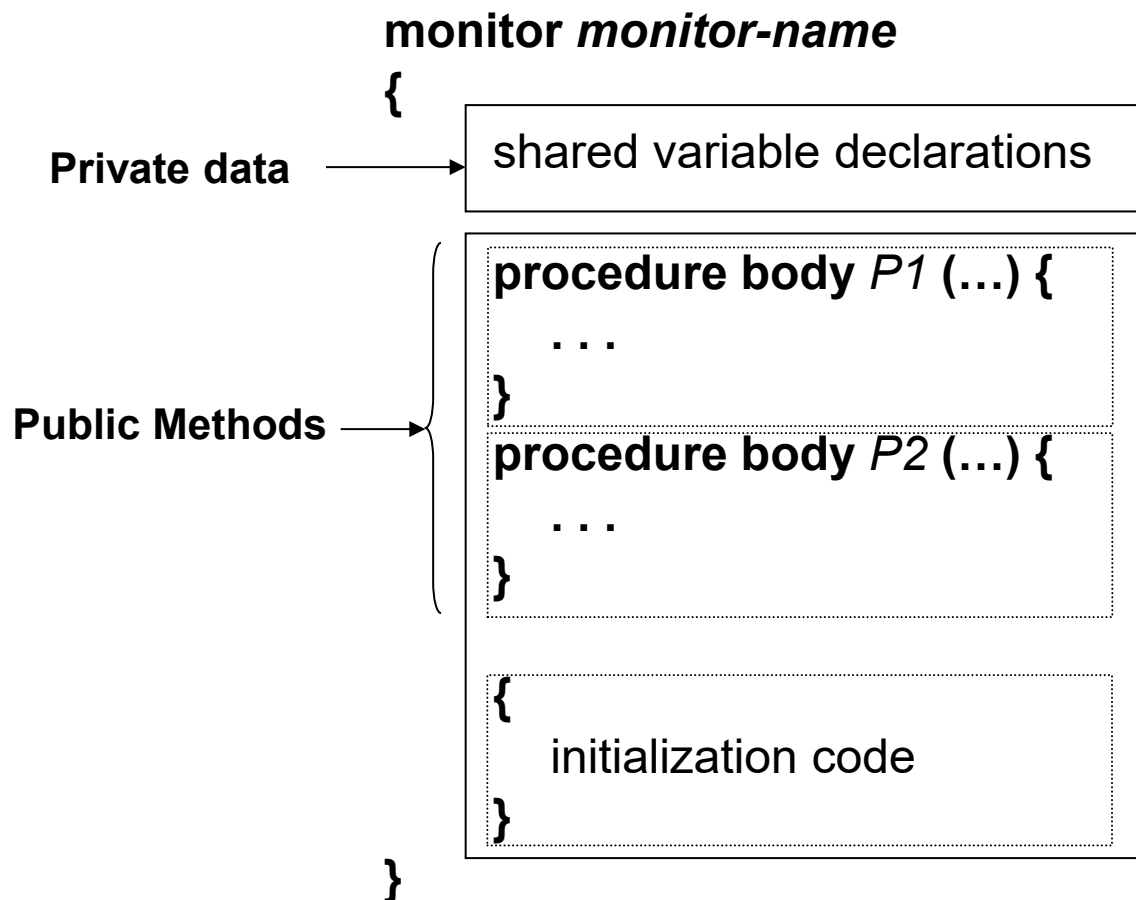


Problems of Semaphore

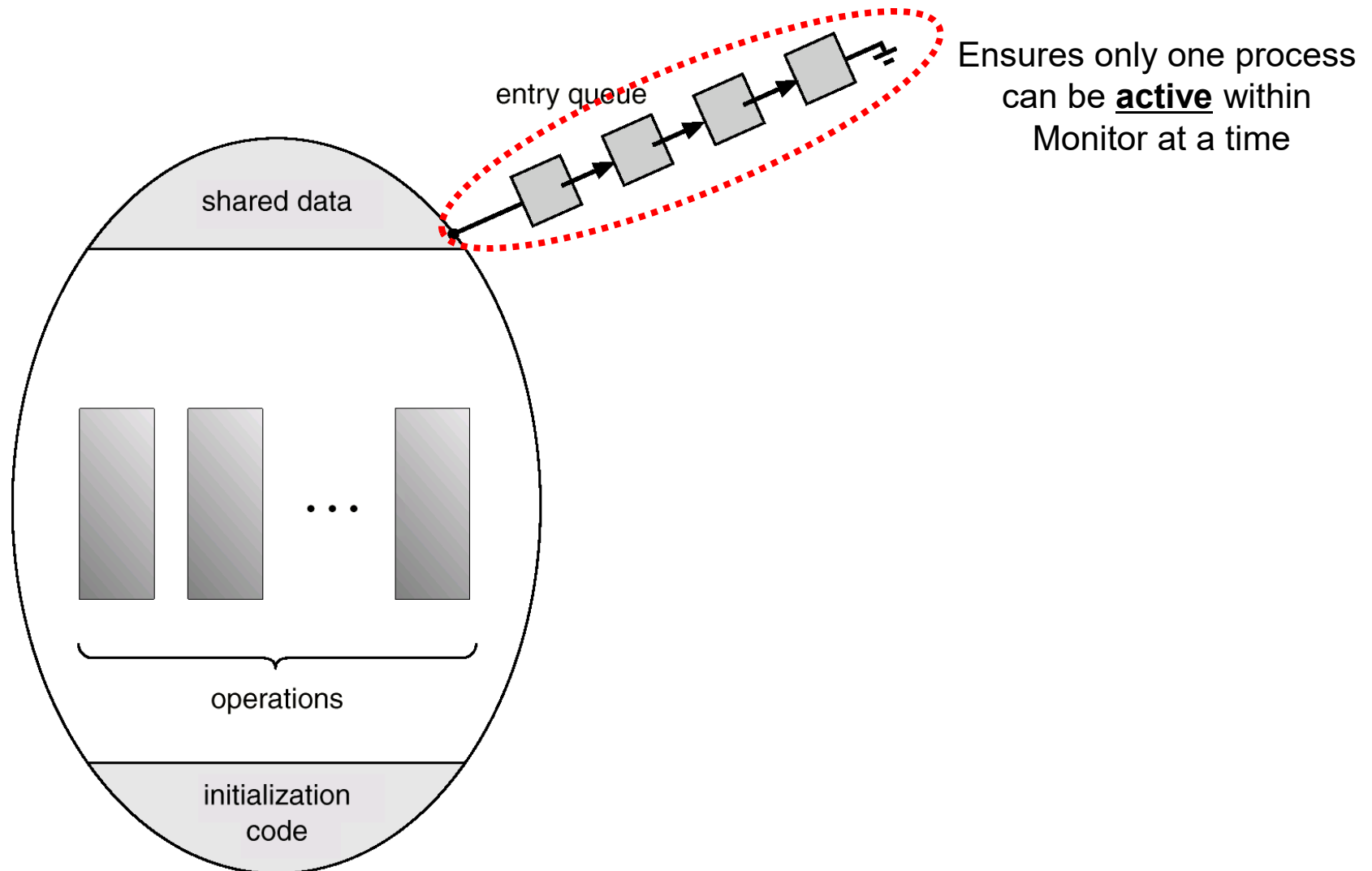
- Difficult to code
- Difficult to prove correctness
 - errors are not reproducible
 - error are observed rarely
- Requires voluntary cooperation
- Single misuse → affect entire system

Monitors: Similar to Abstract data type (private data -- public methods)

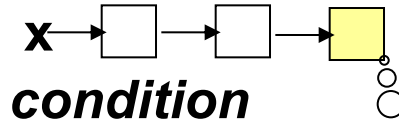
- *High-level language synchronization construct* that allows the **safe sharing** of an abstract data type



Schematic View of a Monitor



Monitors



Insert: **x.wait**
Delete: **x.signal**

- To allow a process to wait within the monitor, a **condition variable** must be declared, as

condition **x, y;**

- Condition variable *can only be used with* the operations **wait** and **signal**.

- The operation

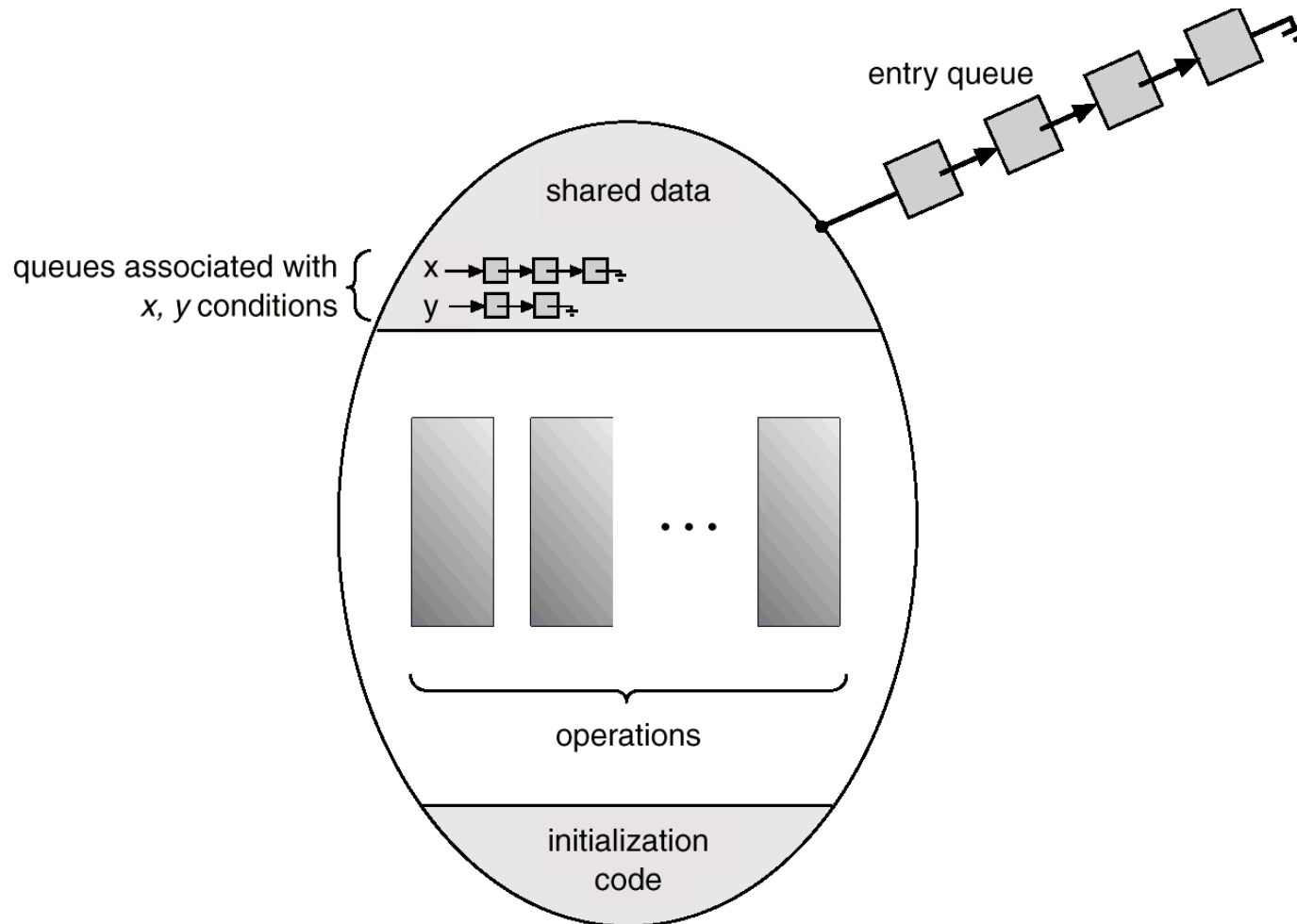
x.wait();

means that the process invoking this operation is suspended until another process invokes

x.signal();

- The **x.signal** operation resumes exactly one suspended process. If no process is suspended, then the **signal** operation has no effect.

Monitor With Condition Variables



Monitor: Dining-Philosophers Problem

monitor dining_philosopher

```

{
    enum {thinking, eating,
        hungry} state[5];
    condition self[5]; /*wait here*/
    void pickup(int i) {
        state[i] = hungry;
        test(i);
        if (state[i] != eating)
            self[i].wait(); /*wait here*/
    }

    void putdown(int i) {
        state[i] = thinking;
        // test left and right neighbors
        test((i+4) % 5); /*if L is waiting*/
        test((i+1) % 5);
    }
}

```

```

void test (int i) {
    if ( (state[(i + 4) % 5] != eating) &&
        (state[i] == hungry) &&
        (state[(i + 1) % 5] != eating)){ /OK/
        state[i] = eating;
        self[i].signal(); /*wakeup Pi */
        /* used for putdown L & R */
    }
}

void init() {
    for (int i = 0; i < 5; i++)
        state[i] = thinking;
}

```

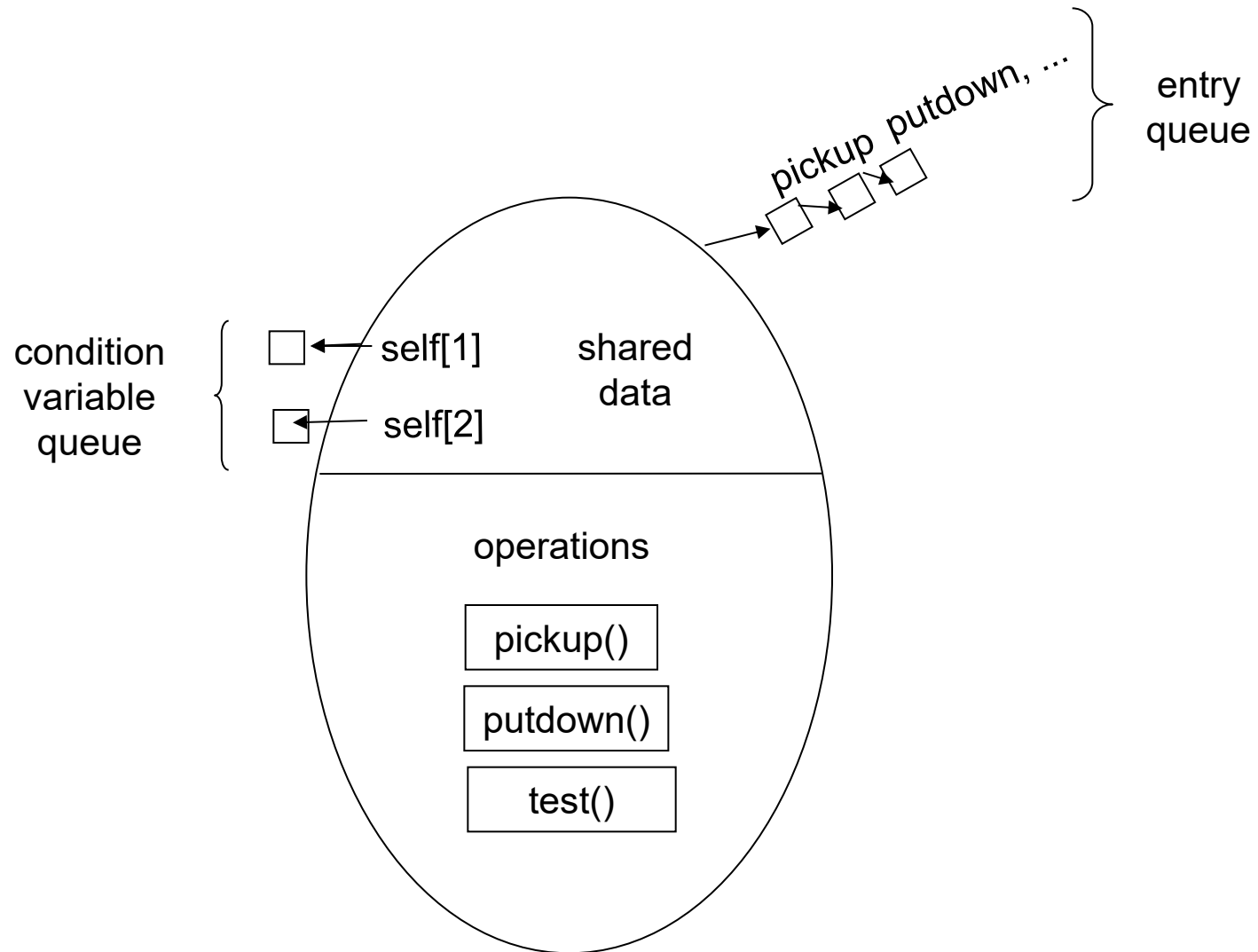
Each Philosopher:

```

{
    pickup(i);
    eat();
    putdown(i);
    think();
} while(1)

```


Monitor: Dining-Philosophers Problem





Monitor Implementation

- *Conditional-wait* construct: **x.wait(c);**
 - **c** – integer expression evaluated when the **wait** operation is executed
 - value of **c** (a *priority number*) stored with the name of the process that is suspended
 - when **x.signal** is executed, process with smallest associated priority number is resumed next
- Check two conditions to establish correctness of system:
 - User processes must always make their calls on the monitor in a correct sequence
 - Must ensure that an uncooperative process does not ignore the mutual-exclusion gateway provided by the monitor, and try to access the shared resource directly, without using the access protocols